Clase 9 (Unidad 4) — Tiempo real con Django Channels y WebSockets

Duración total: 4 horas | Proyecto: primer canal de notificaciones por WebSocket

Cómo leer esta guía

Cada parte combina dos tipos de contenido:

- **Conceptos** (qué es y por qué): la base teórica de la Unidad 4, con tablas y diagramas. Se pueden leer antes de la clase.
- **Práctica** (cómo): los pasos con código, que aplican esos conceptos a TaskFlow.

Al final hay **preguntas de repaso** para autoevaluarse.

Parte 0 — Antes de empezar (5 min)

Parados en `taskflow/`, entorno virtual activo. Cerramos la Unidad 3 con una API segura, autenticada y con permisos por rol. Hoy cambiamos de paradigma: dejamos, por un momento, el modelo petición-respuesta de HTTP.

Parte 1 — Por qué tiempo real: polling vs. WebSockets (55 min)

1.1 El límite de HTTP tradicional

HTTP es un protocolo de **petición-respuesta**: el cliente siempre inicia la comunicación, el servidor solo responde. Si queremos que el navegador se entere "en vivo" de que otro usuario creó una tarea en el mismo proyecto, con HTTP puro solo tenemos estas opciones:

- **Polling** (sondeo): el cliente pregunta cada N segundos "¿hay novedades?", las haya o no. Simple de implementar, pero desperdicia muchísimas peticiones vacías y agrega hasta N segundos de demora en enterarse de un cambio real.
- **Long-polling**: el servidor mantiene la conexión HTTP abierta (sin responder todavía) hasta tener novedades reales o hasta un timeout. Mejor latencia que el polling simple, pero sigue siendo pesado en cantidad de conexiones abiertas simultáneamente.

Hagan la cuenta: 200 usuarios con la app abierta haciendo polling cada 5 segundos son 40 peticiones por segundo contra el servidor, aunque nadie cambie nada.

Concepto: las cuatro técnicas de "tiempo real" en la web. Además del polling y el long-polling, existen los **Server-Sent Events** (SSE) y los WebSockets:

Técnica	Quién inicia	Dirección	Conexión	Cuándo conviene
Polling	El cliente, cada N segundos	Cliente → servidor	Una petición nueva cada vez	Datos que cambian poco; máxima simplicidad
Long-polling	El cliente; el servidor demora la respuesta	Cliente → servidor	Una petición larga, que se repite	Cuando no hay WebSockets disponibles
Server-Sent Events (SSE)	El servidor, sobre una respuesta HTTP que nunca termina	Solo servidor → cliente	Una, persistente	Notificaciones, cotizaciones, texto que genera una IA de a poco
WebSocket	Cualquiera de los dos, en cualquier momento	**Bidireccional**	Una, persistente	Chat, edición colaborativa, juegos, presencia

Dos ideas para comparar las técnicas:

- **Latencia:** cuánto tarda en llegar un cambio. En el polling, hasta N segundos; en SSE y WebSockets, prácticamente nada.

- **Sobrecarga** (*overhead*): cuánto se gasta además del dato útil. Cada petición HTTP lleva cientos de bytes de headers; un mensaje de WebSocket, apenas unos pocos bytes de encabezado.

Para TaskFlow, donde queremos avisos del servidor **y** mensajes del cliente (por ejemplo, "estoy escribiendo"), la opción natural es WebSocket. La tarea de la Clase 10 retoma la comparación con SSE.

1.2 WebSockets

WebSocket es un protocolo **distinto** de HTTP que abre una conexión **bidireccional y persistente**: una vez establecida, tanto el cliente como el servidor pueden enviar mensajes en cualquier momento, sin repetir todo el ciclo de petición-respuesta para cada mensaje. El *handshake* inicial sí es una petición HTTP normal, que luego se "actualiza" (*upgrade*) a WebSocket.

Concepto: el handshake. Así se ve el intercambio que abre la conexión (con los valores del ejemplo del estándar RFC 6455):

```
Cliente → servidor (una petición HTTP GET común, con headers especiales)
  GET /ws/projects/1/ HTTP/1.1
  Host: 127.0.0.1:8000
  Upgrade: websocket
  Connection: Upgrade
  Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
  Sec-WebSocket-Version: 13

Servidor → cliente
  HTTP/1.1 101 Switching Protocols
  Upgrade: websocket
  Connection: Upgrade
  Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+xOo=
```

- `Upgrade: websocket` pide cambiar de protocolo sobre la misma conexión.
- `101 Switching Protocols` es el único código 1xx que van a ver seguido: significa "de acuerdo, a partir de ahora hablamos WebSocket". Si el servidor no acepta (por ejemplo, un 404 o un 403), no hay conexión.
- `Sec-WebSocket-Accept` se calcula a partir de `Sec-WebSocket-Key` con una fórmula fija. Sirve para confirmar que el servidor realmente entiende WebSocket, no para dar seguridad.
- Como el handshake es HTTP, viaja con las cookies y pasa por los mismos proxies: por eso podemos autenticar al usuario en ese momento (Clase 10).

Concepto: mensajes y cierre. Después del handshake, la comunicación se hace en **marcos** (*frames*): de texto (lo que usamos, con JSON adentro), binarios (archivos, audio), de control (`ping` y `pong`, para comprobar que la conexión sigue viva) y de cierre. El cierre lleva un **código**: `1000` es un cierre normal, `1006` indica que la conexión se cortó sin aviso (se cayó la red), y del `4000` al `4999` quedan libres para que cada aplicación defina los suyos.

Concepto: consecuencias de una conexión persistente. Una conexión que queda abierta durante horas cambia la forma de pensar el servidor:

- Cada conexión ocupa memoria mientras dure: 10.000 usuarios conectados son 10.000 conexiones abiertas a la vez. Esto es la **gestión de conexiones concurrentes** del programa.
- El servidor tiene que saber **quién** está conectado **dónde**, para poder avisarle: eso es el *channel layer* y sus grupos (Parte 3.1 y Clase 10).
- Las conexiones se cortan: por la red, por un reinicio del servidor o por límites de la plataforma (Vercel las corta a los 5 minutos). El cliente tiene que estar preparado para **reconectar**.

> 💡 **Nota del docente**
> El protocolo WebSocket en sí (definido en el estándar RFC 6455) es agnóstico al lenguaje de programación: no es "de Python" ni "de Django". Lo que sí es específico de nuestro stack es **cómo lo implementamos**: en Django, a través de **Django Channels**, un paquete que complementa el modelo de servidor tradicional (WSGI: sincrónico, una petición, una respuesta y listo) con **ASGI**, que sí admite conexiones de larga duración como las de WebSocket.

Parte 2 — Configuración de Django Channels, paso a paso (75 min)

2.1 Instalación

Parados en `taskflow/`, entorno virtual activo:

```
pip install "channels[daphne]"
pip freeze > requirements.txt
```

Instalamos dos cosas: **Channels** (la librería que agrega WebSockets a Django) y **Daphne**, un servidor **ASGI**. Es necesario porque el `runserver` de siempre es un servidor WSGI: sabe responder peticiones HTTP, pero no mantener conexiones WebSocket abiertas. Cuando `daphne` está instalado y registrado como app, el comando `runserver` pasa a usar Daphne por debajo, y así el mismo comando de siempre sirve HTTP y WebSocket a la vez.

2.2 Migrar el proyecto a ASGI

Concepto: WSGI y ASGI. Son dos **interfaces estándar** entre el servidor web y la aplicación Python, del mismo modo que un enchufe estándar permite conectar cualquier artefacto a cualquier toma:

	WSGI	ASGI
Modelo	Sincrónico: una petición entra, se procesa y sale	Asincrónico: una conexión puede quedar abierta y recibir eventos
La aplicación es	Una función que recibe la petición y devuelve la respuesta	Una función `async` que recibe `scope` (datos de la conexión), `receive` (para esperar eventos) y `send` (para enviarlos)
Protocolos	Solo HTTP	HTTP, WebSocket y otros
Servidores	Gunicorn, uWSGI, el `runserver` clásico	Daphne, Uvicorn, Hypercorn

Django soporta los dos: `config/wsgi.py` y `config/asgi.py` existen desde la Clase 1.

Concepto: concurrencia con async. Un servidor puede atender muchas conexiones a la vez de dos formas:

- **Hilos o procesos:** cada conexión ocupa un hilo o un proceso mientras dure. Funciona bien con peticiones cortas, pero no escala para miles de conexiones que pasan casi todo el tiempo esperando.
- **Bucle de eventos** (*event loop*, lo que usa `async`): un solo hilo atiende muchas conexiones, pasando de una a otra cada vez que alguna tiene que **esperar** algo (la red, la base de datos). Es ideal para tareas que esperan mucho y calculan poco (**limitadas por E/S**), como un WebSocket que pasa horas esperando mensajes.

La regla de oro del código asincrónico: **nunca bloquear el bucle de eventos**. Si dentro de un `async def` se ejecuta algo lento y sincrónico (por ejemplo, una consulta del ORM de Django, que es sincrónico), todas las conexiones de ese proceso quedan esperando. Por eso Channels ofrece `database_sync_to_async`, que ejecuta el código sincrónico en otro hilo y deja libre el bucle (tarea de hoy y Clase 10).

Práctica.

En `taskflow/config/settings.py`, agreguen `"daphne"` como **primer** elemento de la lista `INSTALLED_APPS` (tiene que quedar arriba de `django.contrib.staticfiles`, porque reemplaza su versión de `runserver`):

```
INSTALLED_APPS = [
    "daphne",
    "django.contrib.admin",
    "django.contrib.auth",
    # ... el resto como estaba
]
```

Y al final del archivo:

```
# --- Tiempo real (Clase 9) ---
INSTALLED_APPS += ["channels"]
ASGI_APPLICATION = "config.asgi.application"

CHANNEL_LAYERS = {
    "default": {
```

```
        "BACKEND": "channels.layers.InMemoryChannelLayer",
    },
}
```

- `ASGI_APPLICATION` le dice a Django (y a Channels) dónde encontrar el punto de entrada de la aplicación asincrónica: la variable `application` del archivo `config/asgi.py`. Django generó ese archivo en la Clase 1, aunque hasta ahora no lo habíamos tocado.
- `CHANNEL_LAYERS` configura el **channel layer**: el "pizarrón" compartido donde las conexiones se anotan en **grupos** y se pasan mensajes entre sí. Hoy usamos la versión en memoria (`InMemoryChannelLayer`), que alcanza para desarrollar con un solo proceso. En la Clase 10 la reemplazamos por Redis.

💡 **Nota del docente**
Channels **no** trae un channel layer configurado por defecto. Si se olvidan del bloque `CHANNEL_LAYERS`, el WebSocket falla al conectar con `AttributeError: 'NoneType' object has no attribute 'group_add'`: `self.channel_layer` vale `None` porque no hay ningún pizarrón configurado.

Abran `taskflow/config/asgi.py` y **reemplacen** su contenido por:

```
import os
from django.core.asgi import get_asgi_application

os.environ.setdefault("DJANGO_SETTINGS_MODULE", "config.settings")
django_asgi_app = get_asgi_application()

from channels.routing import ProtocolTypeRouter, URLRouter
from channels.auth import AuthMiddlewareStack
import core.routing

application = ProtocolTypeRouter({
    "http": django_asgi_app,
    "websocket": AuthMiddlewareStack(
        URLRouter(core.routing.websocket_urlpatterns)
    ),
})
```

💡 **Nota del docente**
Es importante llamar a `get_asgi_application()` **antes** de importar `core.routing` (que a su vez importa `core/consumers.py`, que suele importar modelos): así Django ya está completamente inicializado cuando el resto del código intenta usarlo. Si invierten el orden, aparece el error `AppRegistryNotReady: Apps aren't loaded yet`, difícil de diagnosticar. Por eso esos imports quedan "en el medio" del archivo, contra la costumbre de poner todos los imports arriba.

`ProtocolTypeRouter` es el "portero" que decide según el tipo de conexión entrante: si es HTTP normal, la manda al Django de siempre (`django_asgi_app`, todo lo que ya construimos); si es un WebSocket, la manda al enrutador de Channels (`URLRouter`, con las rutas que vamos a crear en la Parte 3). `AuthMiddlewareStack` completa `self.scope["user"]` dentro de un consumer, de forma parecida a `request.user` en una vista HTTP. Por ahora lo hace a partir de la **sesión** del navegador; en la Clase 10 le sumamos JWT.

2.3 Correr con un servidor ASGI

El comando es el mismo de siempre:

```
python manage.py runserver
```

La diferencia está en la salida de la consola. Verifiquen que diga:

```
Starting ASGI/Daphne version 4.x.x development server at http://127.0.0.1:8000/
```

Si en cambio dice `Starting development server at ...` sin mencionar ASGI/Daphne, `"daphne"` no está primero en `INSTALLED_APPS` (o no se instaló), y los WebSockets van a responder `404`.

(En producción, Clase 12, lanzamos Daphne directamente en lugar de `runserver`, que **no** está pensado para producción bajo ningún concepto, ni con Channels ni sin él.)

Parte 3 — El primer Consumer y su routing (85 min)

3.1 ¿Qué es un Consumer?

Un **Consumer** es, para WebSockets, el equivalente a una vista para HTTP: recibe eventos de conexión y mensajes entrantes, y decide qué hacer con cada uno. La diferencia clave: una vista vive lo que dura una petición; un consumer vive **mientras la conexión esté abierta** (pueden ser horas).

Concepto: el ciclo de vida de una conexión.

```
cliente abre ws://.../ws/projects/1/
   ▼
connect()        → decidir si se acepta (accept) o se rechaza (close); unirse a grupos
   ▼
receive() ...    → se ejecuta por cada mensaje que manda el cliente (cero, una o muchas veces)
métodos de grupo → se ejecutan por cada mensaje que llega al grupo (Clase 10)
   ▼
disconnect()     → la conexión se cerró, por cualquier motivo: limpiar (salir de los grupos)
```

Concepto: canales, grupos y "hubs". El programa habla de "implementación de hubs y canales de comunicación". Cada tecnología usa su nombre para las mismas ideas:

Idea	Django Channels	Socket.IO (jueves)	SignalR (.NET)
El punto de entrada que atiende las conexiones	Consumer	Servidor `io` o *namespace*	**Hub**
La dirección única de cada conexión	`channel_name`	`socket.id`	`ConnectionId`
Un conjunto de conexiones que reciben lo mismo	**Grupo**	**Sala** (*room*)	**Grupo**
Enviar a todos los de un conjunto	`group_send`	`io.to(sala).emit(...)`	`Clients.Group(...)`
Lo que comparte los mensajes entre procesos	*Channel layer* (Redis)	*Adapter* (Redis)	*Backplane* (Redis)

Entender el concepto permite pasar de un stack a otro: en todos, un **grupo** por proyecto es la forma de mandar un aviso solo a los interesados.

Práctica.

Creen el archivo `taskflow/core/consumers.py`:

```
import json
from channels.generic.websocket import AsyncWebsocketConsumer


class TaskNotificationConsumer(AsyncWebsocketConsumer):
    async def connect(self):
        self.project_id = self.scope["url_route"]["kwargs"]["project_id"]
        self.group_name = f"project_{self.project_id}"

        # Nos unimos a un "grupo" (lo profundizamos en la Clase 10)
        await self.channel_layer.group_add(self.group_name, self.channel_name)
        await self.accept()

    async def disconnect(self, close_code):
        await self.channel_layer.group_discard(self.group_name, self.channel_name)

    async def receive(self, text_data):
        data = json.loads(text_data)
        # Por ahora, hacemos "echo" del mensaje recibido
        await self.send(text_data=json.dumps({"echo": data}))
```

```
    async def task_notification(self, event):
        # Este método es llamado cuando llega un mensaje al grupo (Clase 10)
        await self.send(text_data=json.dumps(event["payload"]))
```

Método por método:

- `connect` se ejecuta automáticamente cuando un cliente **intenta** abrir la conexión. `self.scope` es un diccionario con toda la información de esa conexión (similar al `request` de una vista HTTP). `self.scope["url_route"]["kwargs"]["project_id"]` extrae el parámetro `project_id` que definimos en el routing (Parte 3.2).
- `self.channel_layer.group_add(self.group_name, self.channel_name)` anota esta conexión en el grupo `project_<id>` del pizarrón. `self.channel_name` es el "número de casilla" único que Channels le asigna a cada conexión. Es la base del broadcast de la Clase 10: todas las conexiones al mismo proyecto quedan en el mismo grupo.
- `await self.accept()` es imprescindible: sin esta línea, la conexión queda pendiente y el cliente nunca llega a establecerla.
- `disconnect` se ejecuta cuando la conexión se cierra (por cualquier motivo) y saca a la conexión del grupo.
- `receive` se ejecuta cada vez que llega un mensaje **desde** el cliente ya conectado. Hoy simplemente lo devolvemos tal cual ("echo"), a modo de prueba. `json.loads` convierte el texto recibido en un diccionario de Python; `json.dumps` hace lo inverso para enviarlo.
- `task_notification` todavía no se usa: lo va a llamar Channels en la Clase 10, cuando alguien publique un mensaje en el grupo.

3.2 Routing de WebSockets

Creen el archivo `taskflow/core/routing.py`:

```
from django.urls import re_path
from . import consumers

websocket_urlpatterns = [
    re_path(r"ws/projects/(?P<project_id>\d+)/$", consumers.TaskNotificationConsumer.as_asgi()),
]
```

- `re_path` (a diferencia del `path` que veníamos usando para HTTP) acepta expresiones regulares completas; es lo que se usa tradicionalmente en el routing de Channels.
- `(?P<project_id>\d+)` es un **grupo con nombre** (`project_id`) que captura uno o más dígitos (`\d+`) de la URL: exactamente el valor que después leemos en `self.scope["url_route"]["kwargs"]["project_id"]`.
- `$` marca el final de la URL: `ws/projects/1/` coincide, `ws/projects/1/otra-cosa/` no.
- `.as_asgi()` convierte la clase del consumer en algo que el enrutador de Channels puede usar directamente (equivalente a `.as_view()` en las vistas basadas en clase de Django).
- Por convención, las URLs de WebSocket empiezan con `ws/`, para distinguirlas a simple vista de las de la API (`api/`).

3.3 Probar la conexión

Instalen una librería para probar WebSockets desde Python (solo la usamos para probar; no hace falta en el servidor):

```
pip install websocket-client
```

Con el servidor corriendo (`python manage.py runserver`, en una terminal), abran **otra** terminal (con el entorno virtual activo también ahí) y ejecuten:

```
python -c "
import websocket
ws = websocket.create_connection('ws://127.0.0.1:8000/ws/projects/1/')
ws.send('{\"mensaje\": \"hola desde el cliente\"}')
```

```
print(ws.recv())
ws.close()
"
```

`ws://` (en vez de `http://`) es el esquema de URL del protocolo WebSocket sin cifrar. El equivalente cifrado, análogo a `https://`, es `wss://`, el que usaríamos en producción. Deberían ver impreso en la terminal el mensaje de "echo" que devuelve el consumer:

```
{"echo": {"mensaje": "hola desde el cliente"}}
```

Y en la terminal del servidor, las líneas `WebSocket HANDSHAKING`, `WebSocket CONNECT` y `WebSocket DISCONNECT`. También pueden probar con una extensión de navegador como "Simple WebSocket Client", apuntando a la misma URL `ws://127.0.0.1:8000/ws/projects/1/`.

💡 **Nota del docente**
`AsyncWebsocketConsumer` usa `async`/`await`: de esta forma, Python puede manejar miles de conexiones simultáneas dentro de un mismo proceso, porque mientras una conexión está "esperando" algo (por ejemplo, datos de la red), el proceso puede atender a otra en ese tiempo muerto. Es una diferencia clave frente al modelo sincrónico de las vistas HTTP que veníamos usando desde la Clase 1, donde cada petición ocupa el proceso de punta a punta.

💡 **Nota del docente**
Noten que hoy **cualquiera** puede conectarse a cualquier proyecto, sin token. El WebSocket todavía no aplica nada de lo que construimos en la Unidad 3. Lo resolvemos en la Clase 10: no dejen un WebSocket así en producción.

Parte 4 — Guardar el avance en Git y GitHub (20 min)

```
git status
git add .
git commit -m "Migra el proyecto a ASGI con Django Channels y agrega el primer consumer de WebSocket"
git push
```

Parte 5 — Cierre y tarea

5.1 Resumen

- Diferencia entre polling, long-polling y WebSockets, y por qué HTTP puro no alcanza para tiempo real.
- Migración del proyecto a ASGI: `daphne` en `INSTALLED_APPS`, `config/asgi.py` y el `ProtocolTypeRouter` que distingue HTTP de WebSocket.
- El channel layer y los grupos, configurados en memoria.
- Primer `Consumer` asincrónico con `connect`/`disconnect`/`receive`, y su routing con `re_path` + `.as_asgi()`.

5.2 Tarea para la casa

1. Modifiquen el consumer para que, al conectar, rechace la conexión (`await self.close()`) si el `project_id` recibido no existe en la base de datos. Van a necesitar `database_sync_to_async` (de `channels.db`) para consultar el ORM de forma segura desde código `async`, porque el ORM de Django es sincrónico. Por ejemplo: `existe = await database_sync_to_async(Project.objects.filter(id=self.project_id).exists)()`.
2. Lean el cuestionario de la Unidad 4 que les entregó el docente, en particular la pregunta 2 sobre el handshake de WebSocket (RFC 6455): la discutimos al inicio de la próxima clase.
3. Guarden el avance repitiendo el flujo de la Parte 4.

5.3 Próxima clase

Clase 10: **grupos, broadcast, presencia de usuarios, autenticación del WebSocket y Redis como channel layer**, la parte que hace que el tiempo real escale a múltiples usuarios y procesos.

Errores frecuentes

- **`Handshake status 404 Not Found`**: el servidor no está corriendo con Daphne. Revisen que la consola diga `Starting ASGI/Daphne` (Parte 2.3) y que la URL termine en `/` (`ws/projects/1/`).
- **`Handshake status 500` y en el servidor `'NoneType' object has no attribute 'group_add'`**: falta el bloque `CHANNEL_LAYERS` en `settings.py`.
- **`AppRegistryNotReady: Apps aren't loaded yet`**: en `asgi.py`, `import core.routing` quedó arriba de `get_asgi_application()`.
- **`ModuleNotFoundError: No module named 'websocket'`**: instalaron `websocket-client` con el entorno virtual desactivado, o en la otra terminal no está activado.
- **`ConnectionRefusedError`**: el servidor no está corriendo, o está en otro puerto.

Preguntas de repaso

1. Comparen polling, long-polling, SSE y WebSocket en dirección de la comunicación, latencia y sobrecarga.
2. Describan el handshake de WebSocket. ¿Qué significa el código `101`?
3. ¿Por qué se puede autenticar al usuario en el momento del handshake?
4. ¿Qué es un marco (*frame*)? ¿Para qué sirven `ping` y `pong`? ¿Qué indica el código de cierre `1006`?
5. ¿Qué cambia en el servidor cuando las conexiones duran horas en vez de milisegundos?
6. ¿Qué diferencia hay entre WSGI y ASGI? ¿Por qué `runserver` solo no alcanza para WebSockets?
7. ¿Qué es el bucle de eventos? ¿Por qué es ideal para tareas limitadas por E/S?
8. ¿Qué significa "no bloquear el bucle de eventos"? ¿Para qué sirve `database_sync_to_async`?
9. Describan el ciclo de vida de un consumer.
10. ¿Qué es un grupo? ¿Cómo se llama el mismo concepto en Socket.IO y en SignalR?

Para profundizar

- RFC 6455, *The WebSocket Protocol*: `rfc-editor.org/rfc/rfc6455`
- MDN, *WebSockets API*: `developer.mozilla.org/es/docs/Web/API/WebSockets_API`
- Documentación de Django Channels: `channels.readthedocs.io`
- Especificación ASGI: `asgi.readthedocs.io`
- MDN, *Server-Sent Events*: `developer.mozilla.org/es/docs/Web/API/Server-sent_events`

Diccionario de términos de esta clase

- **`async`/`await`:** palabras clave de Python para escribir código asincrónico, que puede "pausarse" mientras espera algo (como datos de red) sin bloquear todo el proceso.
- **ASGI (Asynchronous Server Gateway Interface):** interfaz estándar, sucesora de WSGI, que permite a Django manejar conexiones de larga duración como WebSocket.
- **Channel layer:** mecanismo de Channels que permite que distintas conexiones (y procesos) se pasen mensajes, organizadas en grupos.
- **Bucle de eventos (event loop):** mecanismo que atiende muchas tareas en un solo hilo, alternando entre ellas mientras esperan.
- **Channel name:** dirección única que Channels asigna a cada conexión.
- **Consumer:** en Django Channels, la clase equivalente a una vista, pero para conexiones WebSocket.
- **Daphne:** servidor ASGI oficial del proyecto Channels.
- **Frame (marco):** unidad en la que viajan los mensajes de un WebSocket (texto, binario, control o cierre).
- **Grupo (hub, sala):** conjunto de conexiones que reciben los mismos mensajes.
- **Handshake (apretón de manos):** intercambio inicial entre cliente y servidor para establecer una conexión; en este caso, para pasar de HTTP a WebSocket.

- **Limitada por E/S (I/O bound):** tarea que pasa la mayor parte del tiempo esperando la red o el disco, no calculando.
- **Long-polling:** técnica en la que el cliente pregunta por novedades y el servidor demora la respuesta hasta tenerlas o hasta un timeout.
- **Polling (sondeo):** técnica en la que el cliente pregunta por novedades a intervalos regulares.
- **Routing (en Channels):** mapa que asocia URLs de WebSocket con Consumers, análogo a `urls.py` para HTTP.
- **SSE (Server-Sent Events):** técnica en la que el servidor envía eventos al cliente por una respuesta HTTP que queda abierta; solo en un sentido.
- **WebSocket:** protocolo (RFC 6455) que habilita una conexión bidireccional y persistente entre cliente y servidor.
- **WSGI (Web Server Gateway Interface):** interfaz estándar tradicional entre un servidor web y una aplicación Python sincrónica.